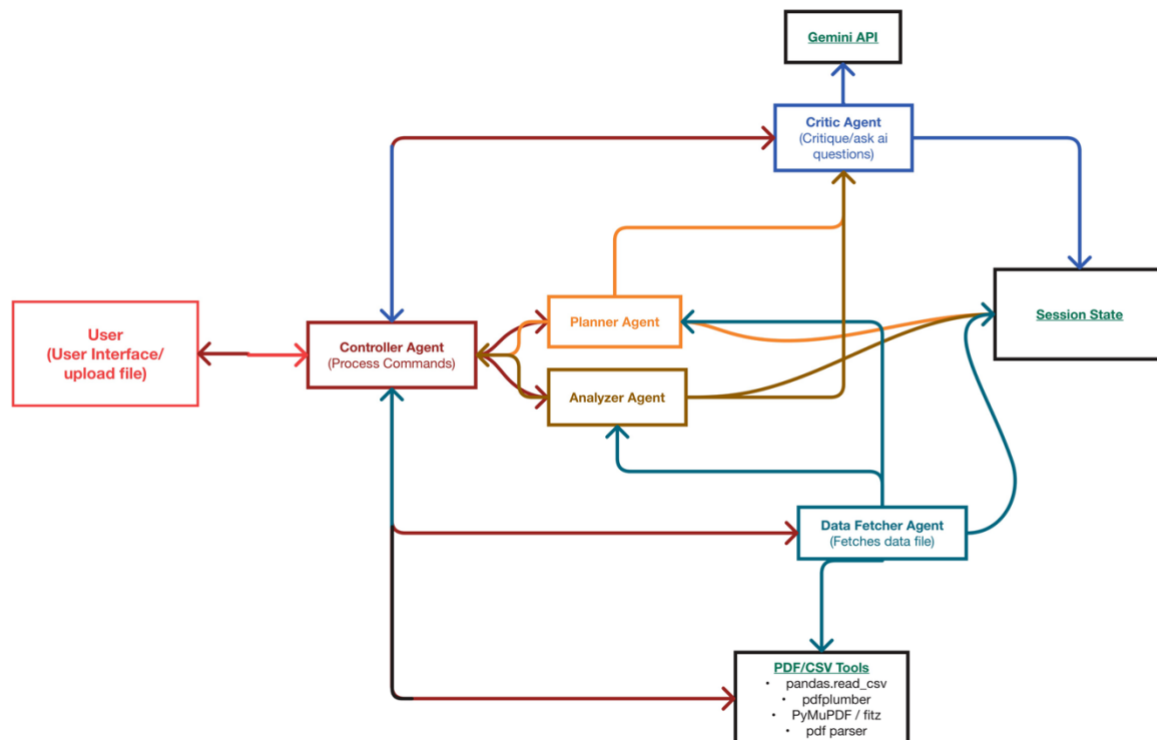


## Milestone II- Multi Agent Workflow

### 1. Image of the Agent Architecture Diagram



Each agent's role and input/output format.

**1. Controller Agent**

Roles: The Controller Agent acts as the central hub of the multi-agent system. It is responsible for parsing user queries, determining the appropriate workflow, dispatching tasks to specialized agents, managing the flow of information between agents, and consolidating final responses for the user. Query parsing, workflow orchestration, task delegation, response aggregation, state management.

Input: User queries, outputs from other agents.

Output: Instructions to specialized agents, consolidated user responses.

**2. Data Fetcher Agent**

Roles: Securely retrieves and prepares financial data from various sources. Connects to bank APIs (mock Plaid), processes CSV uploads, handles data validation and initial cleansing.

Input: User credentials, file uploads (CSV, Excel), data retrieval requests from Controller Agent.

Output: Raw or semi-processed financial transaction data (e.g., DataFrames).

**3. Analyzer Agent**

Roles: Processes raw financial data to extract insights, categorize spending, and identify patterns. Applies machine learning classifiers for transaction categorization, uses pandas for data aggregation and statistical analysis, identifies spending trends and anomalies.

Input: Processed transaction data from Data Fetcher Agent.

Output: Categorized transactions, spending patterns, financial insights, analytical reports.

#### **4. Planner Agent**

Roles: Develops personalized financial plans, budgets, and goal trackers based on analyzed data and user objective. Implements optimization algorithms for budget allocation, creates goal-based saving plans, generates financial forecasts.

Input: Analyzed spending patterns from Analyzer Agent, user financial goals, current financial status.

Output: Personalized budget plans, savings goals, financial projections, action recommendations.

#### **5. Critic Agent**

Roles: Evaluates and stress-tests financial recommendations and plans generated by other agents. Performs scenario simulations (e.g., what-if analyses for market changes, unexpected expenses), assesses risk, provides constructive feedback on proposed plans.

Input: Proposed financial plans/recommendations from Planner Agent or Optimizer Agent.

Output: Risk assessments, scenario analysis reports, plan adjustments, feedback for other agents.

### **Coordination Mechanism: Orchestrated by a Controller Agent**

The Controller Agent is the brain of the operation. When a user submits a query:

1. The Controller Agent first interprets the user's intent (e.g., "Analyze my spending," "Create a budget," "Find savings").
2. Dispatch Tasks: Based on the parsed intent, it dispatches the request to one or more specialized agents. For example, a "create a budget" query might first trigger the Data Fetcher Agent (if new data is needed), then the Analyzer Agent, and finally the Planner Agent.
3. Manage Workflow: The Controller Agent monitors the progress of each task, handling dependencies (e.g., Planner Agent can only act after Analyzer Agent has categorized data). It also manages agent communication, ensuring data is passed correctly between them.
4. Consolidate Response: Once the relevant agents have completed their tasks, the Controller Agent collects their outputs, synthesizes them into a coherent and comprehensive response, and presents it to the user.

#### How they communicate?

Communication is synchronous and orchestrated by the controller agent. The user communicates with the controller agent. The controller agent decides which agent to call based on the user input and calls that agent, that could be any one of the 5 I mentioned above. The chosen agent will communicate with the controller agent when it has a return value, and the controller agent will communicate with the user to display the return value.

### How failures or low-quality outputs are handled?

There are error messages put in place to inform the user of missing information or file read errors. The user will see exactly what the agent returns. There will be a validation check between the agents. There will be a retry logic and suggestions on what may be the issue.